



Lecture 6: Tiny:Bit

What is a Microcontroller

- Small, low-cost computer processor
- Low power
- Used in "embedded" applications
 - Appliances
 - Smart home devices
 - Computer peripherals
 - Cars
 - Consumer electronics
 - Medical equipment
 - Just about everywhere
- Easy to interface with sensors, displays, etc.



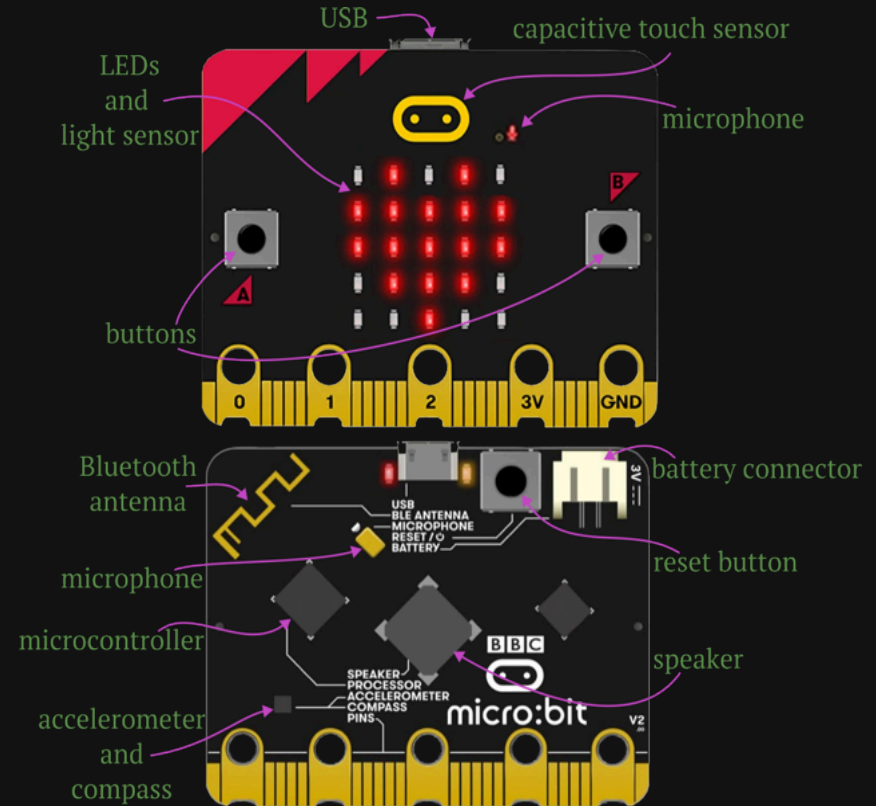
Microcontroller vs Microprocessor

	Microprocessor	Microcontroller
Clock Speed	~2 GHz	1-200 MHz
Number of Cores	4-32	1
Memory	2-256 GB (external)	16-512 kB (internal)
Storage	256-2,000 GB	500-2,000 kB
Power Usage	high to moderate	low to very low
Cost	high to moderate	low to very low
Peripheral Support	Complex, high bandwidth	Simple, low bandwidth



Micro:Bit

- Open-source single board computer (SBC)
- Designed by the BBC for computer education
- Has lots of cool peripherals
 - LED grid
 - Light level sensor
 - Buttons
 - Microphone
 - Speaker
 - Accelerometer (gestures, acceleration)
 - Compass
 - Bluetooth
 - USB
 - Battery connector
- Runs MicroPython!

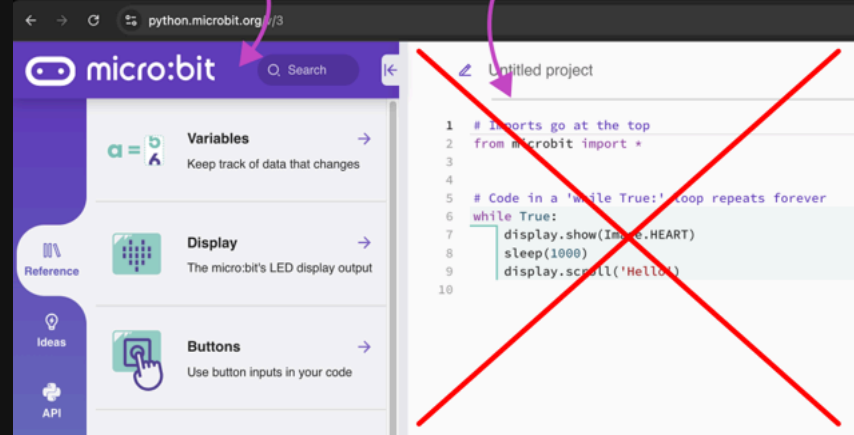




Micro:Bit Editor

- <https://python.microbit.org/v/3>
- Use documentation on Micro:Bit site
- Don't use online editor
 - Not aware of Tiny:Bit hardware
 - Will remove our Tiny:Bit firmware

do use this



don't use this!

MicroPython

- A subset of Python
- Designed to run on microcontrollers
- Processor, memory, and storage constraints
- Everything we've learned so far works
- Except f-strings

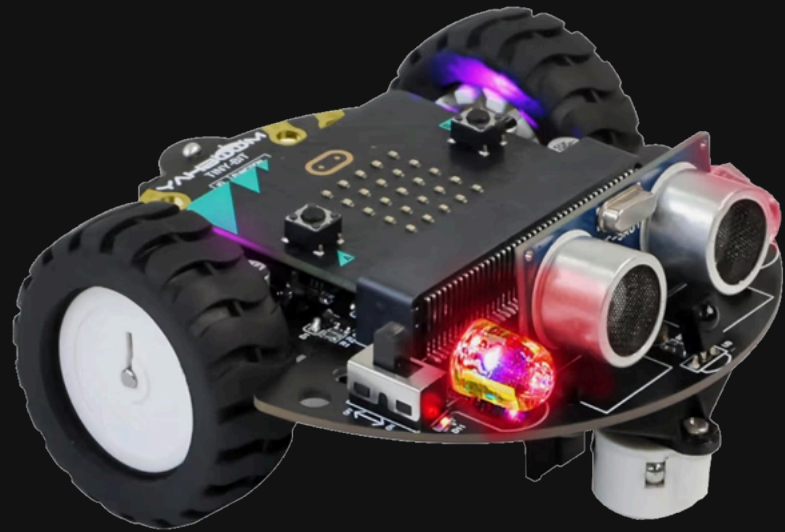
```
# no f-strings in micropython
print(f"{name} has {hp} hit points")

# use string.format instead
print("{} has {} hit points".format(name, hp))

# or just use + to concatenate strings
# remember to use str() to convert non-strings
print(name + " has " + str(hp) + " hit points")
```

Tiny:Bit

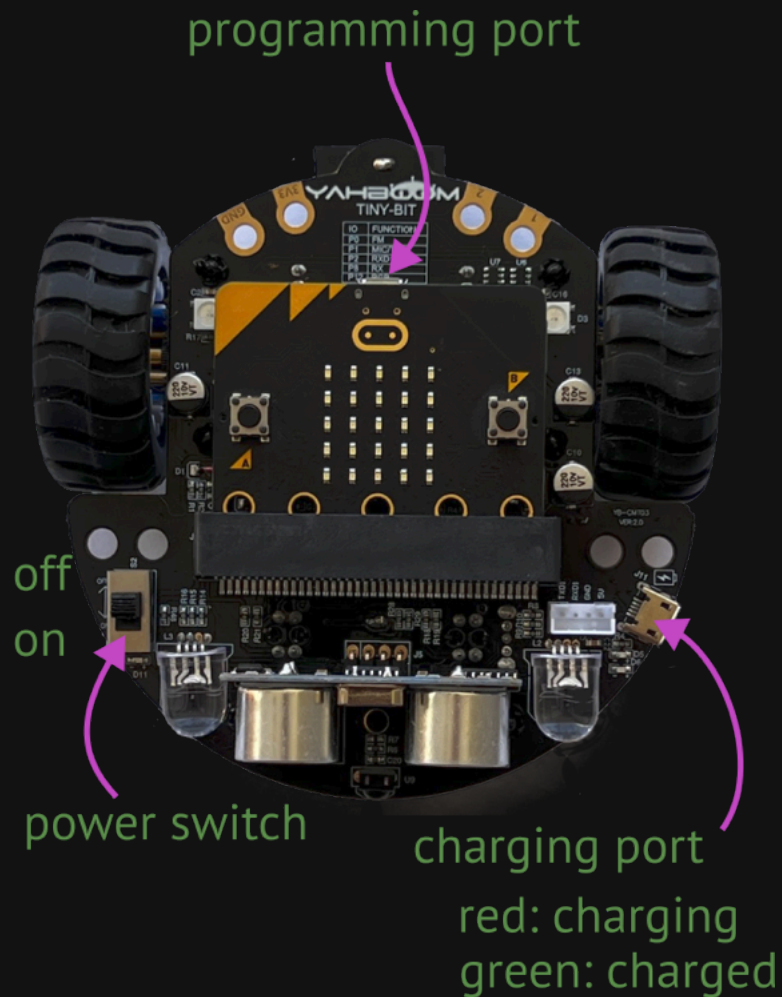
- Motors
- RGB headlights
- RGB taillights
- Ultrasonic distance sensor
- IR receiver and remote
- Line tracking sensors
- Rechargeable battery
- Plus everything on the Micro:Bit!





Tiny:Bit Power

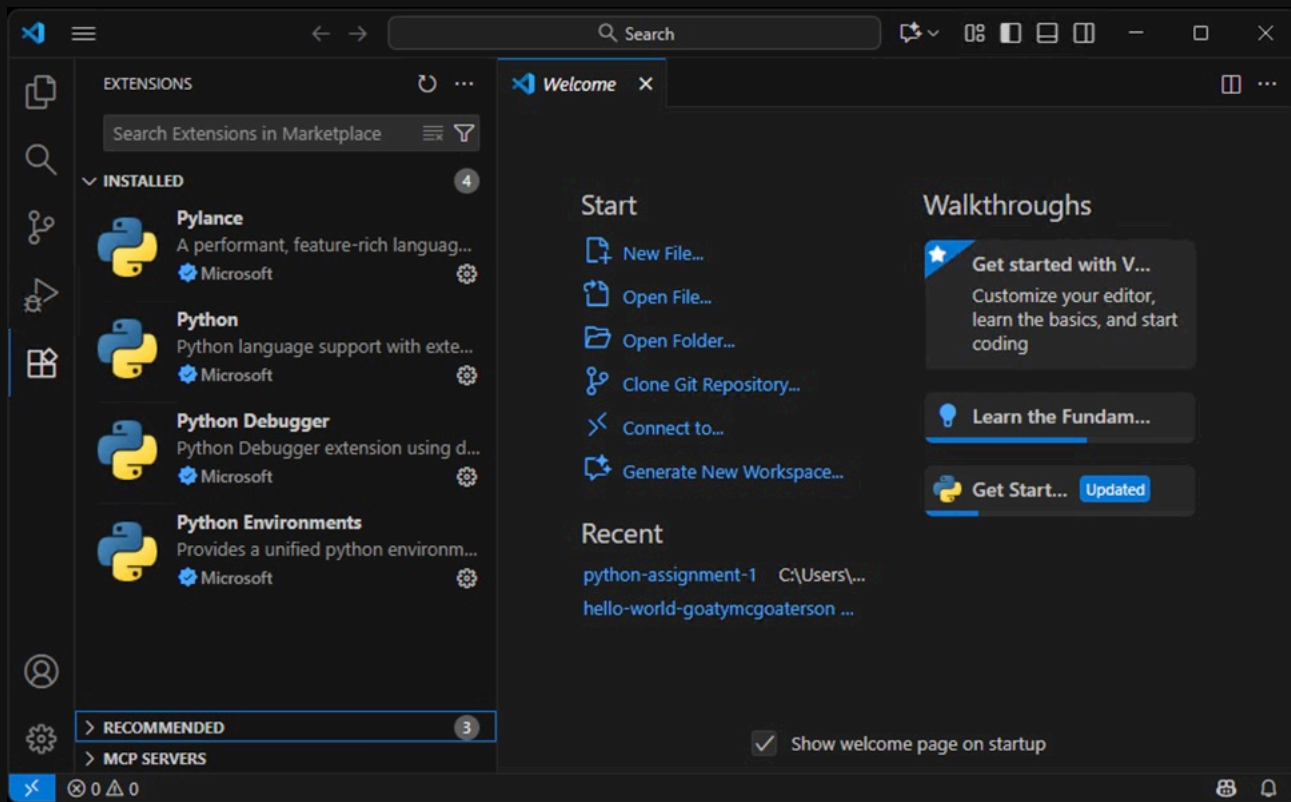
- Use charging port to charge
 - Power switch must be off
- Use Micro:Bit port for programming
 - Be careful unplugging: can brick Micro:Bit
 - *Save before programming!*
 - Your program must be called `main.py`
 - Wait 10 seconds before programming after plugging in
 - Can program with power on or off
 - Some things work poorly if power off



Setting Up Visual Studio Code

- Confirm all our Python extensions are installed
- Install Serial Monitor extension from Microsoft
- Create new project
- Download my `tinybit.zip` file
- Unzip it in your project directory
- Create virtual environment
- Install `microfs2` package in virtual environment
- Flash Micro:Bit with Tiny:Bit firmware
- Put `hybridge_tinybit.py` on Micro:bit
- Create simple `main.py` program
- Put `main.py` on Micro:Bit
- Celebrate! 🎉

🐍 Confirm All our Python Extensions are Installed





Install Serial Monitor from Microsoft

The screenshot shows the Visual Studio Code interface with the Extensions Marketplace open. The search bar contains "serial monitor". The "Serial Monitor" extension by Microsoft is highlighted. The right-hand pane shows the details for the "Serial Monitor" extension, including its icon, name, publisher (Microsoft), and a description: "Send and receive text from serial ports." The extension is currently installed, as indicated by the "Uninstall" button. Below the description, there is a "Get Started" section with instructions on how to use the extension.

EXTENSIONS: MARKETPLACE

serial monitor

Serial Monitor
Send and receive text from serial ports.
Microsoft

Web and Desktop Serial Monitor 52K
Views serial port output on desktop or web
Eclipse CDT Cloud
Install

Close all opened serial ports (vsco... 809 ★ 5
Closes all open serial ports in VS Code Serial Mo...
pietro.cz
Install

Copilot Serial Tool 38 ★ 5
VS Code serial monitor extension with GitHub Co...
Htboss11
Install

ESP Updater 5K
ESP update and monitor tool on Web Serial (Chr...
masuidrive
Install

serial terminal 18K ★ 5
An interactive serial terminal tool
awsxxf
Install

Extension: Serial Monitor

Serial Monitor 
Microsoft  microsoft.com
Send and receive text from s...
Uninstall Switch to Pre-Release Ve

DETAILS FEATURES CHANGELOG

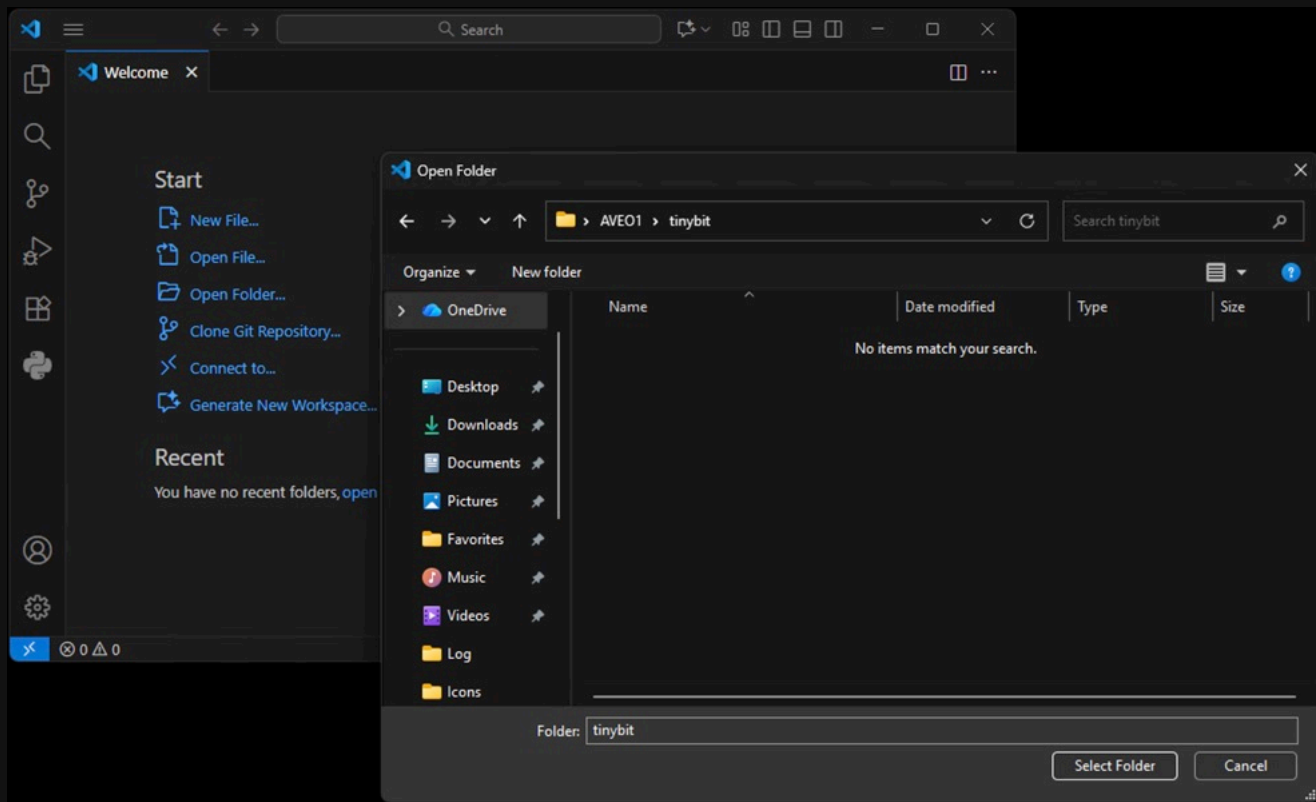
Serial Monitor

The Serial Monitor extension provides a serial monitor to view output from as well as send messages to serial ports. This is often useful when testing or debugging programs on embedded devices.

Get Started

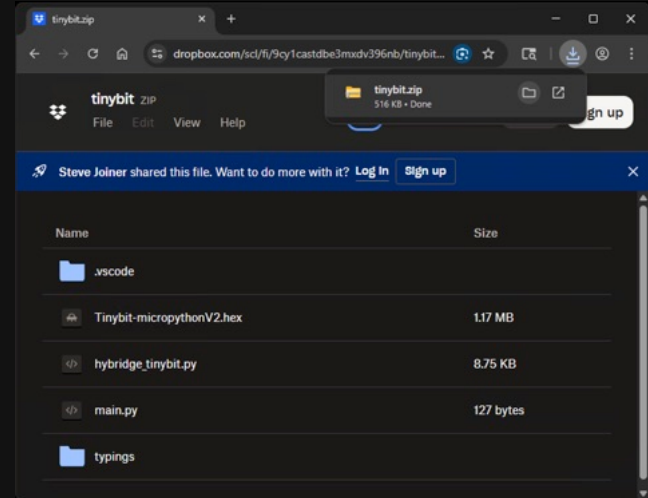
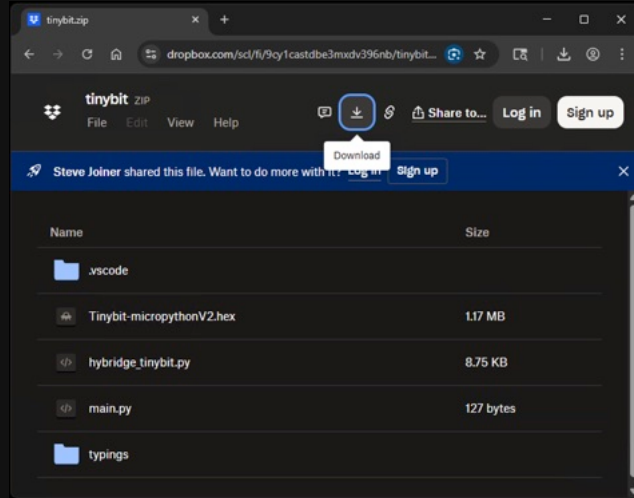
To get started with this extension, you should open ti
Serial Monitor by completing the following steps:

Create new Project



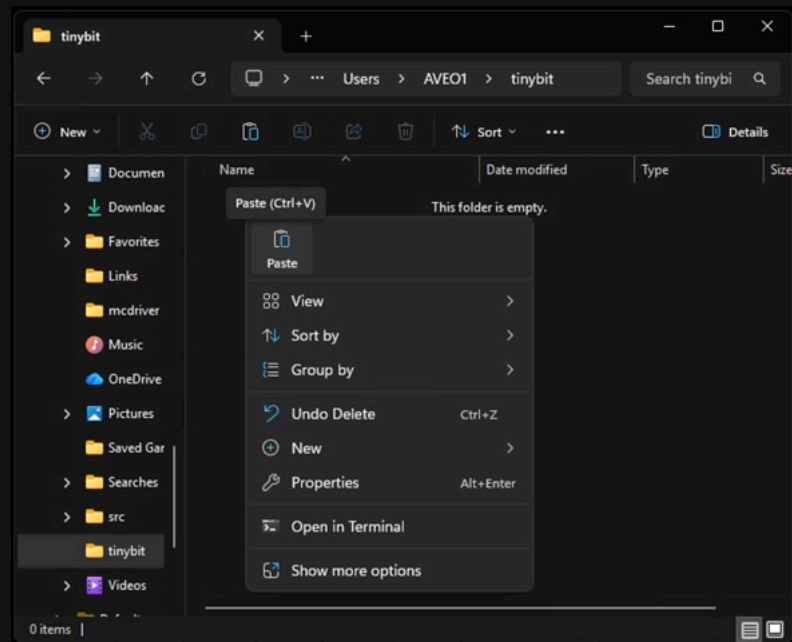
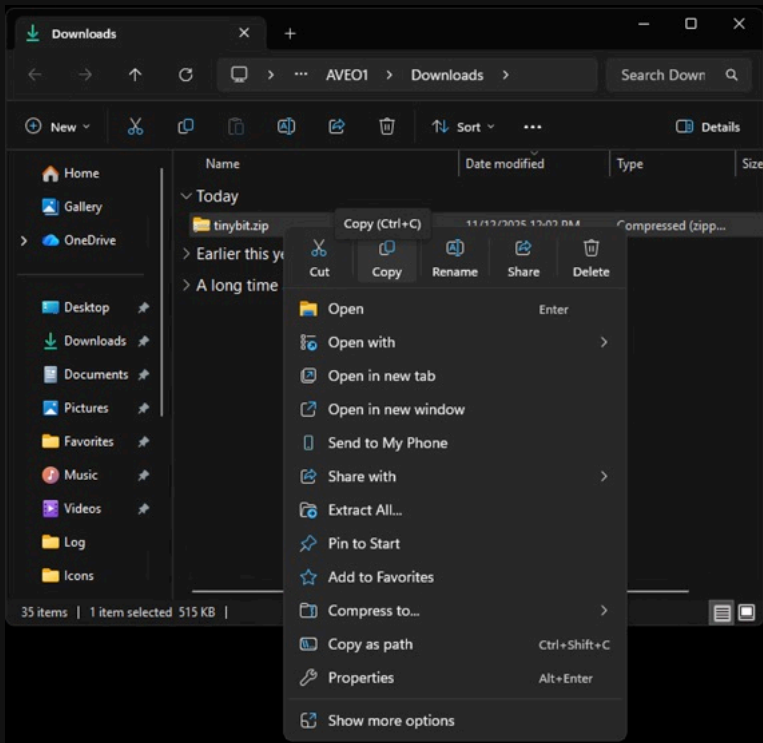
🐍 Download My `tinybit.zip` File

- `tinybit.zip`

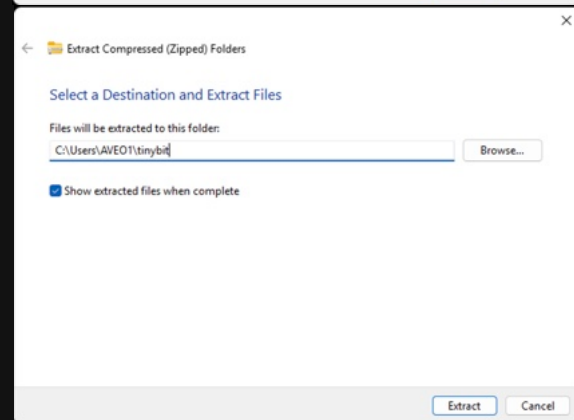
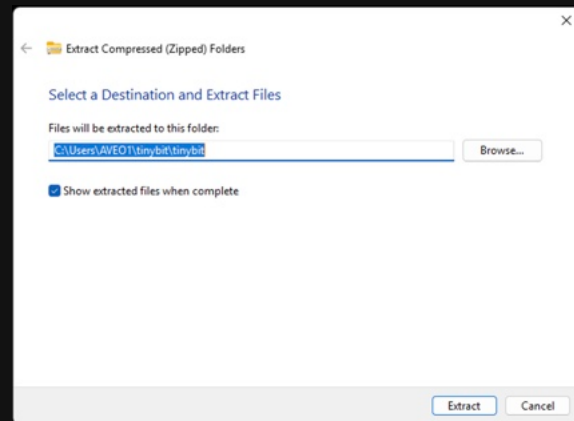
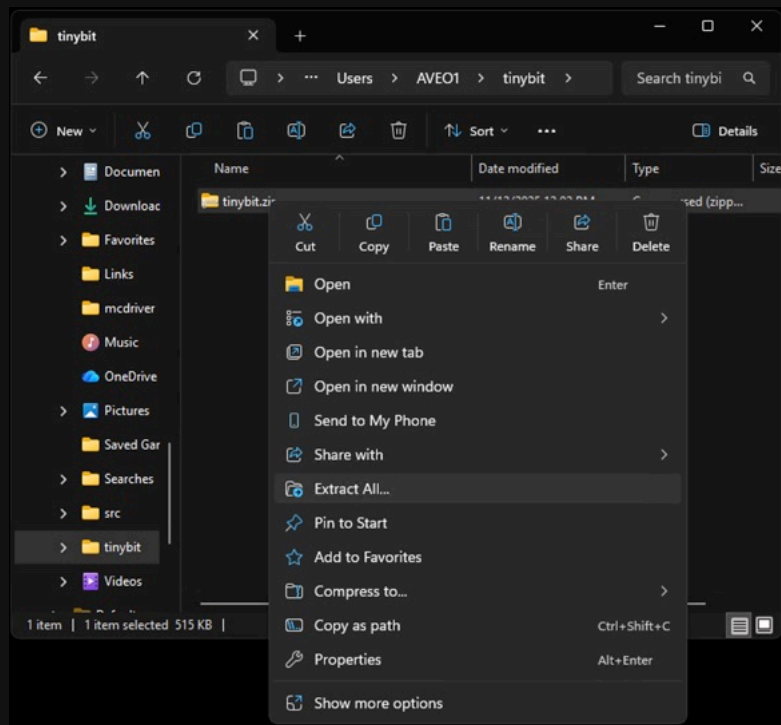




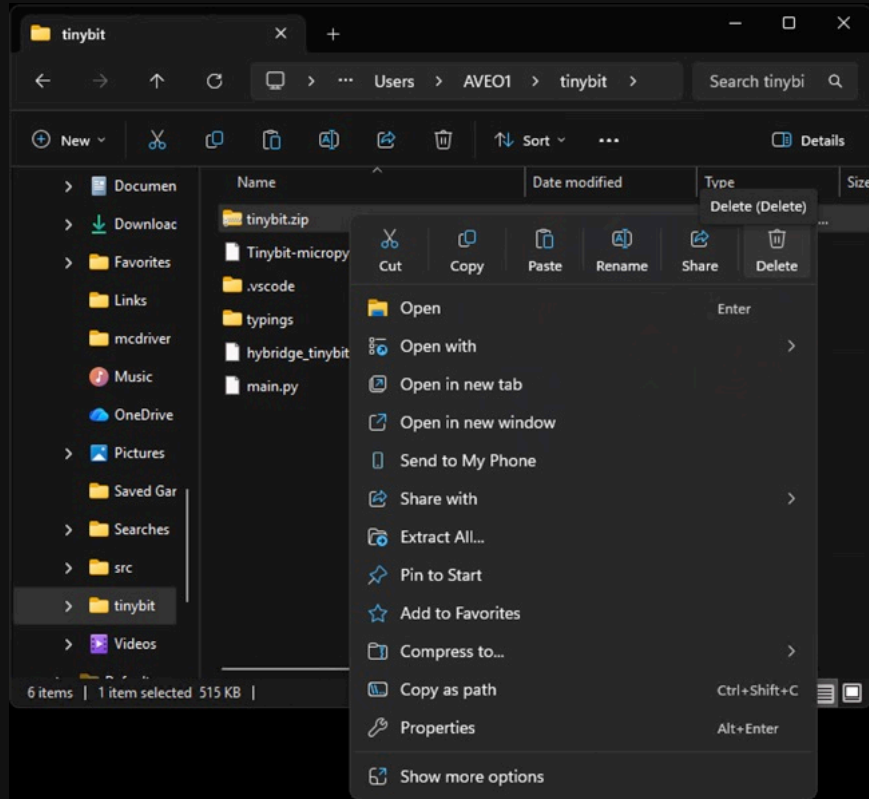
Copy tinybit.zip to Your Project Folder



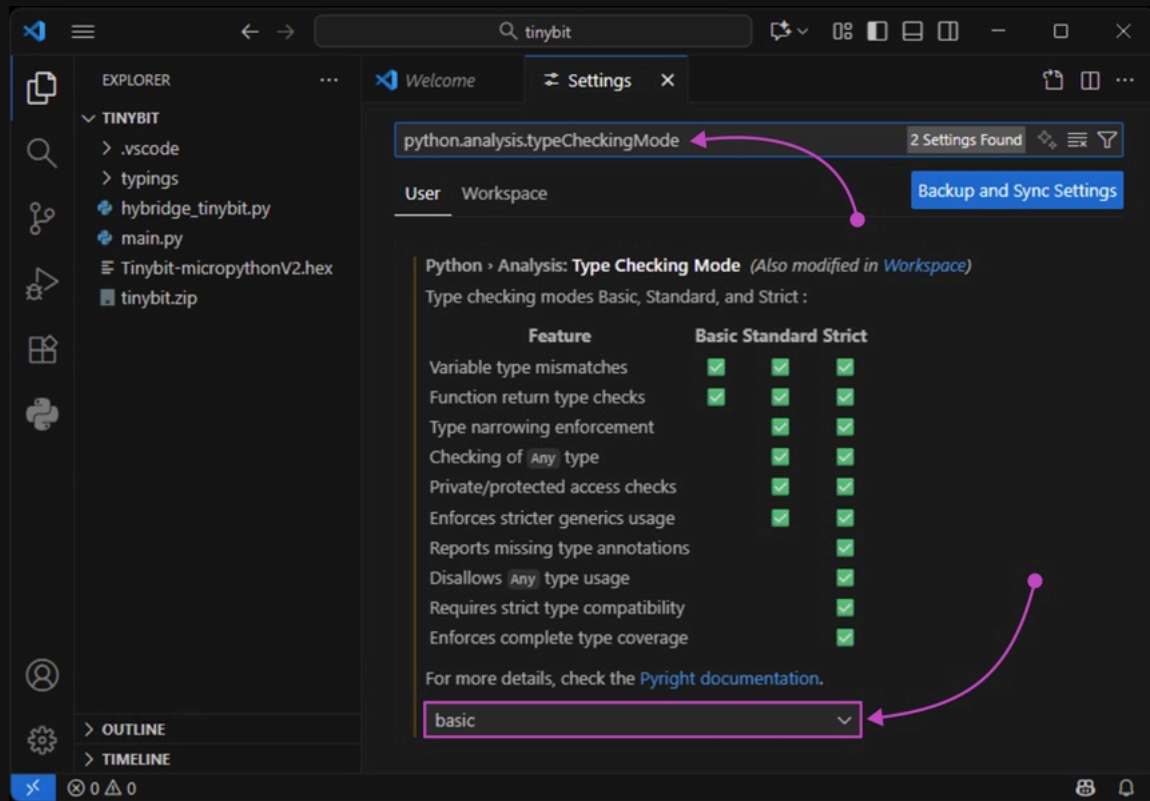
🦩 Extract `tinybit.zip` File in Your Project Folder



🐍 Delete tinybit.zip File in Your Project Folder



🐍 Confirm Pylance is Enabled



The screenshot shows the VS Code interface with the Settings window open. The Explorer sidebar on the left shows a project named 'TINYBIT' with files like '.vscode', 'typings', 'hybride_tinybit.py', 'main.py', 'Tinybit-micropythonV2.hex', and 'tinybit.zip'. The Settings window is titled 'Settings' and shows the search bar with 'python.analysis.typeCheckingMode' entered. A pink arrow points from the search bar to the 'python.analysis.typeCheckingMode' setting. Below the search bar, the 'User' tab is selected, and a 'Backup and Sync Settings' button is visible. The setting is titled 'Python › Analysis: Type Checking Mode (Also modified in Workspace)' and includes a description: 'Type checking modes Basic, Standard, and Strict :'. A table follows, showing the status of various features for each mode. The 'basic' mode is selected in the dropdown at the bottom, indicated by another pink arrow. The status bar at the bottom shows '0 0 0'.

Feature	Basic	Standard	Strict
Variable type mismatches	✓	✓	✓
Function return type checks	✓	✓	✓
Type narrowing enforcement		✓	✓
Checking of <code>Any</code> type		✓	✓
Private/protected access checks		✓	✓
Enforces stricter generics usage		✓	✓
Reports missing type annotations			✓
Disallows <code>Any</code> type usage			✓
Requires strict type compatibility			✓
Enforces complete type coverage			✓



Disable reportMissingModuleSource

The image shows two side-by-side screenshots of the Visual Studio Code interface, illustrating the steps to disable the `reportMissingModuleSource` setting.

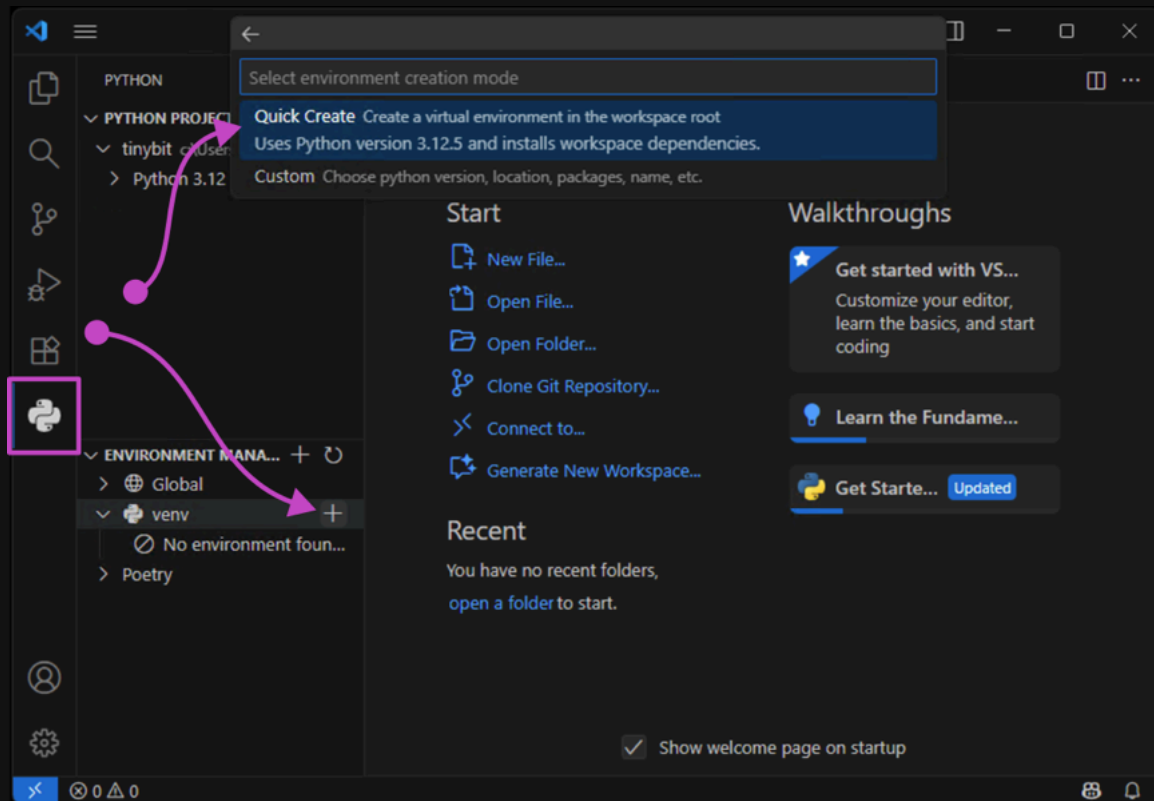
Left Screenshot: The **Settings** window is open, showing the **python.analysis.diagnosticSeverityOverrides** setting. The **Workspace** tab is selected. The description for this setting is visible, and a link to `settings.json` is provided at the bottom.

Right Screenshot: The `settings.json` file is open, showing the configuration for `python.analysis.diagnosticSeverityOverrides`. The `reportMissingModuleSource` property is highlighted with a red box and set to `none`.

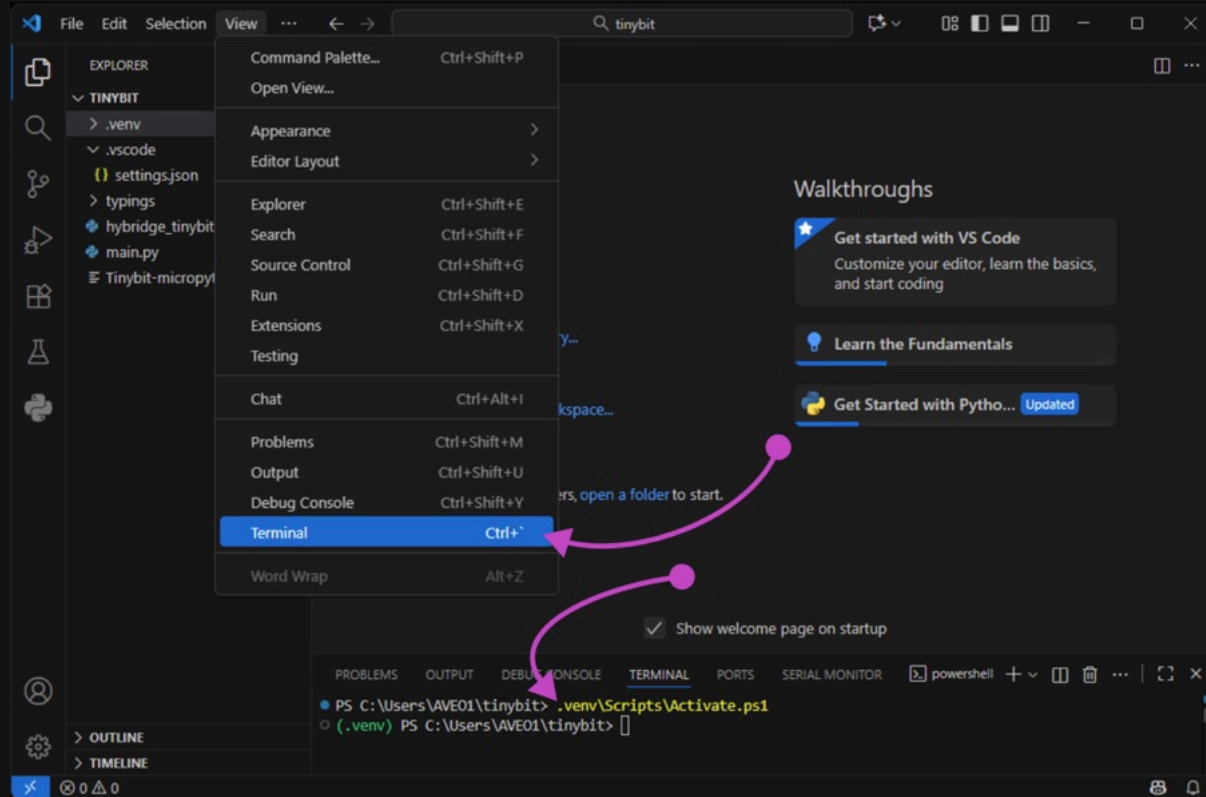
```
1 {  
2   "python.analysis.diagnosticSeverityOverrides": {  
3     "reportMissingModuleSource": "none"  
4   },  
5   "python.analysis.typeCheckingMode": "basic"  
6 }
```

A red arrow points from the `settings.json` file in the right screenshot to the `reportMissingModuleSource` property in the left screenshot, indicating the location of the setting.

Create Virtual Environment



🐍 Activate Virtual Environment in PowerShell



Activate Virtual Environment Error

If you see this:

```
PS C:\Users\AVE01\tinybit> .venv\Scripts\Activate.ps1
.venv\Scripts\Activate.ps1 : File C:\Users\AVE01\tinybit\.venv\Scripts\Activate.ps1 cannot be loaded
because running scripts is disabled on this system. For more information, see about_Execution_Policies at
https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ .venv\Scripts\Activate.ps1
+ ~~~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
```

Then do this:

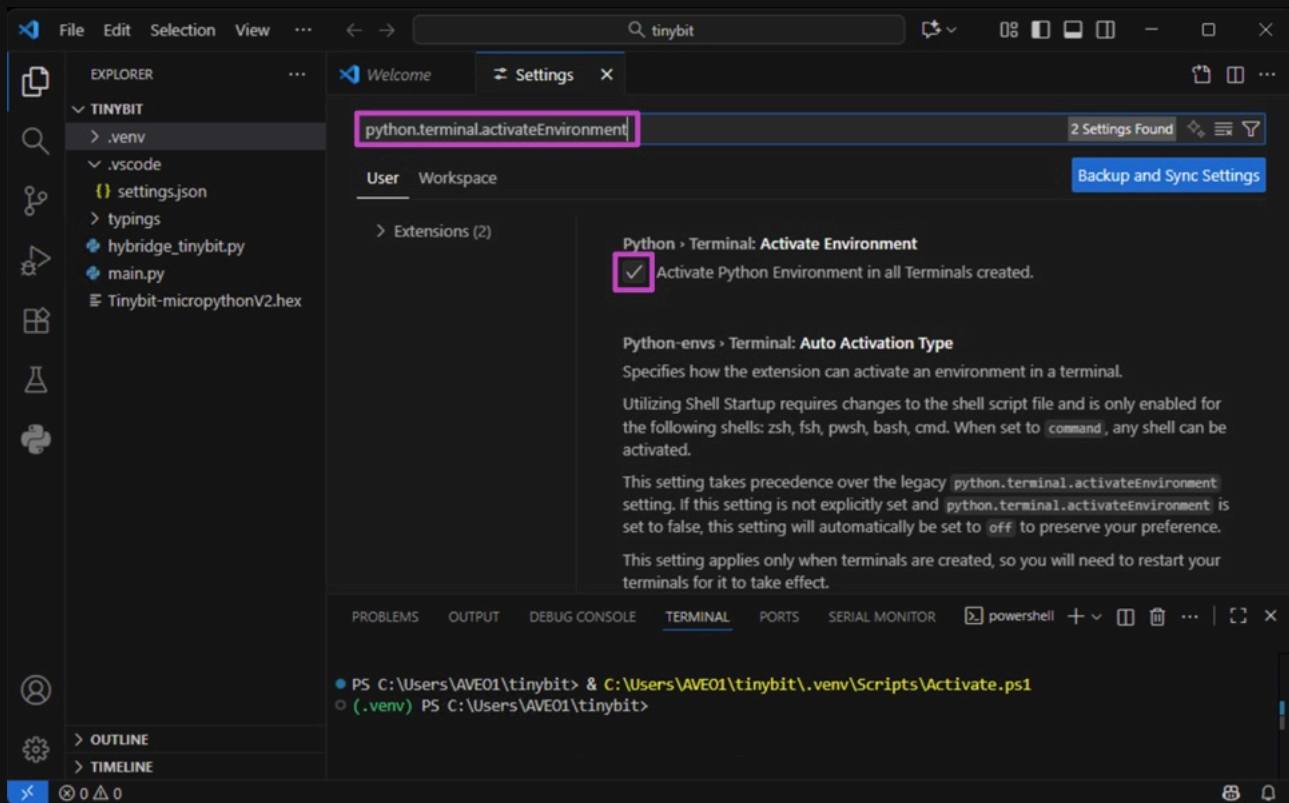
```
PS C:\Users\AVE01\tinybit> Set-ExecutionPolicy Unrestricted -Scope CurrentUser
```

And try again:

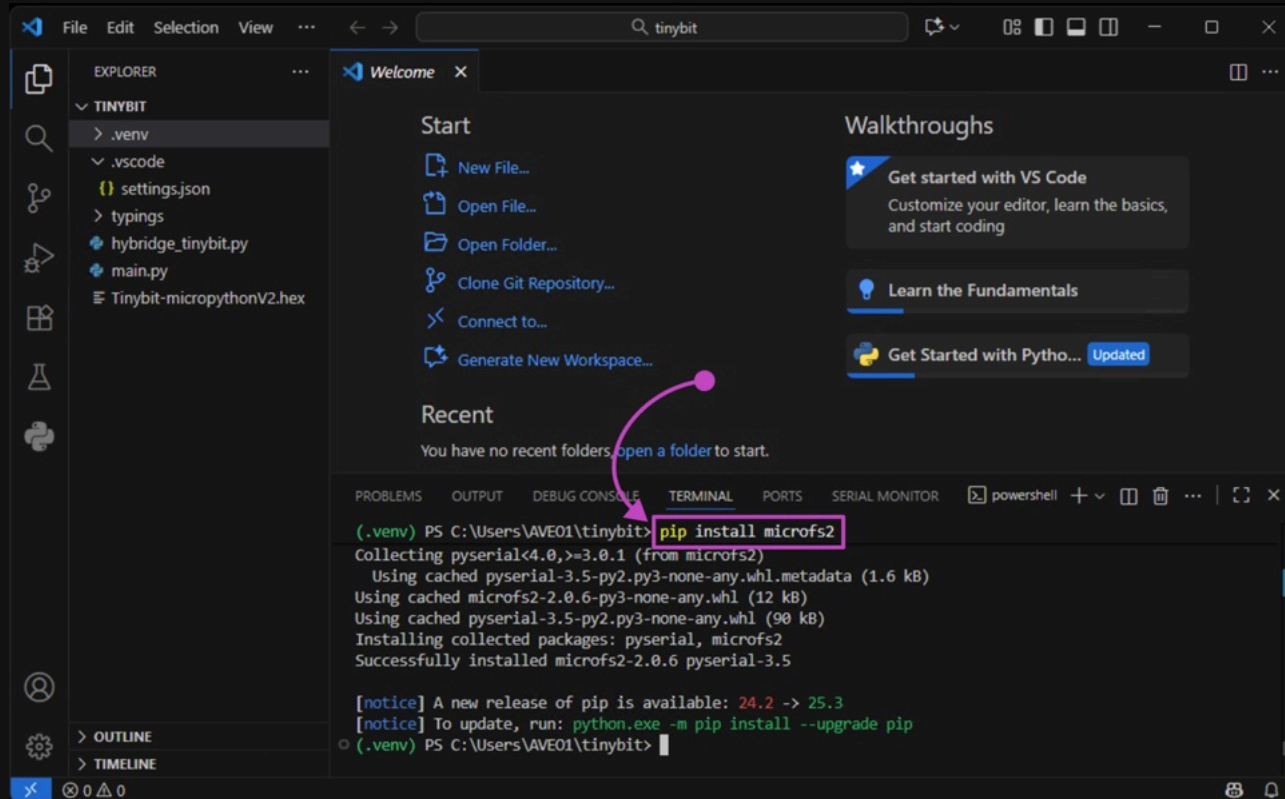
```
PS C:\Users\AVE01\tinybit> .venv\Scripts\Activate.ps1
(.venv) PS C:\Users\AVE01\tinybit> █
```



Automatically Activate Virtual Environment



🐍 Install microfs2 Package in Virtual Environment



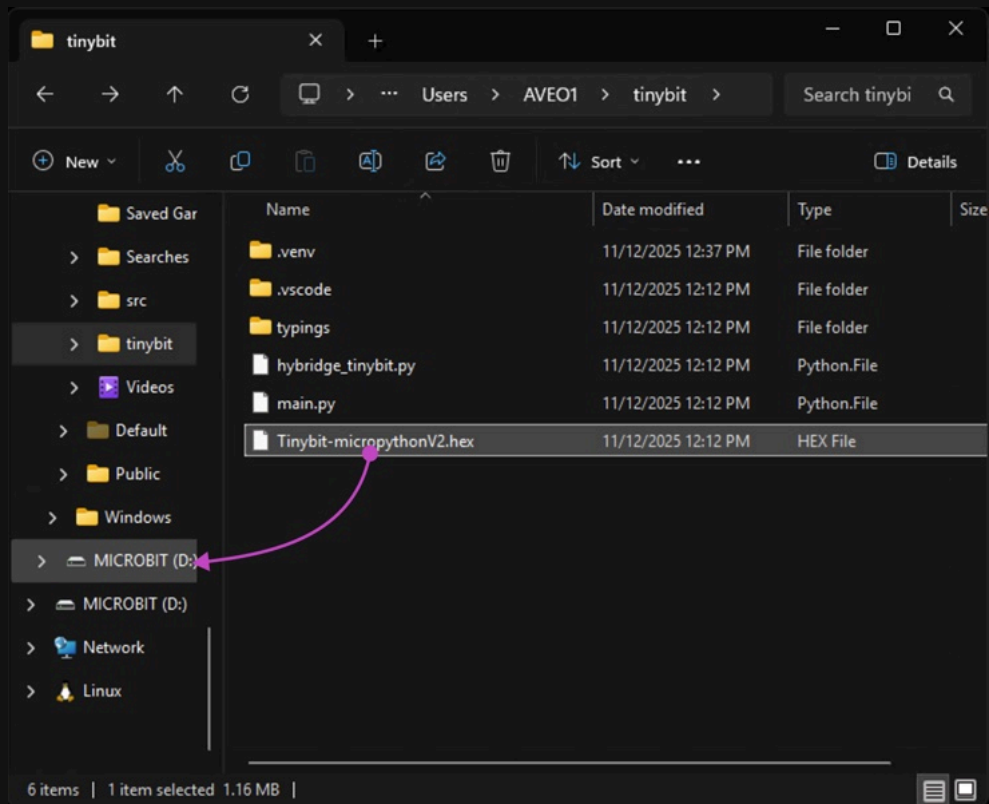
The screenshot shows the Visual Studio Code interface with a project named 'TINYBIT'. The Explorer sidebar on the left shows the file structure: `.venv`, `.vscode`, `settings.json`, `typings`, `hybride_tinybit.py`, `main.py`, and `Tinybit-micropythonV2.hex`. The main editor area displays the 'Welcome' page with options like 'New File...', 'Open File...', 'Open Folder...', 'Clone Git Repository...', 'Connect to...', and 'Generate New Workspace...'. A pink arrow points from the 'Generate New Workspace...' option to the terminal. The terminal window at the bottom shows the command `pip install microfs2` being executed in the `.venv` environment. The output shows that the package is successfully installed, along with some pip update notices.

```
(.venv) PS C:\Users\AVE01\tinybit> pip install microfs2
Collecting pyserial<4.0,>=3.0.1 (from microfs2)
  Using cached pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
Using cached microfs2-2.0.6-py3-none-any.whl (12 kB)
Using cached pyserial-3.5-py2.py3-none-any.whl (90 kB)
Installing collected packages: pyserial, microfs2
Successfully installed microfs2-2.0.6 pyserial-3.5

[notice] A new release of pip is available: 24.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\AVE01\tinybit>
```



Flash Micro:Bit with Tiny:Bit Firmware





Put `hybridge_tinybit.py` on Micro:Bit

The screenshot shows the Visual Studio Code (VS Code) interface. On the left, the Explorer sidebar shows a project named 'TINYBIT' with files: `.venv`, `.vscode`, `settings.json`, `typings`, `hybridge_tinybit.py`, `main.py`, and `Tinybit-micropythonV2.hex`. The main editor area displays the 'Welcome' page with options like 'New File...', 'Open File...', 'Open Folder...', 'Clone Git Repository...', 'Connect to...', and 'Generate New Workspace...'. Below this is the 'Recent' section, which is empty. On the right, there are 'Walkthroughs' for 'Get started with VS Code', 'Learn the Fundamentals', and 'Get Started with Python...'. At the bottom, the TERMINAL panel is active, showing the output of a `pip install microfs2` command. The output indicates that `pyserial` and `microfs2` were successfully installed. A pink arrow points from the `hybridge_tinybit.py` file in the Explorer to the `put hybridge_tinybit.py` command in the terminal.

```
(.venv) PS C:\Users\AVE01\tinybit> pip install microfs2
Collecting pyserial<4.0,>=3.0.1 (from microfs2)
  Using cached pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
  Using cached pyserial-2.0.6-py3-none-any.whl (12 kB)
  Using cached pyserial-3.5-py2.py3-none-any.whl (90 kB)
Installing collected packages: pyserial, microfs2
Successfully installed microfs2-2.0.6 pyserial-3.5

[notice] A new release of pip is available: 24.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\AVE01\tinybit> ufs put hybridge_tinybit.py
(.venv) PS C:\Users\AVE01\tinybit>
```



Create Simple main.py Program

The screenshot shows the Visual Studio Code (VS Code) interface. The Explorer sidebar on the left shows a project named 'TINYBIT' with files: '.venv', '.vscode', 'settings.json', 'typings', 'hybridge_tinybit.py', 'main.py', and 'Tinybit-micropythonV2.hex'. The 'main.py' file is selected and open in the editor. The code in 'main.py' is as follows:

```
1 from microbit import *
2
3 while True:
4     display.show(Image.HEART)
5     sleep(1000)
6     display.show(Image.HAPPY)
7     sleep(1000)
```

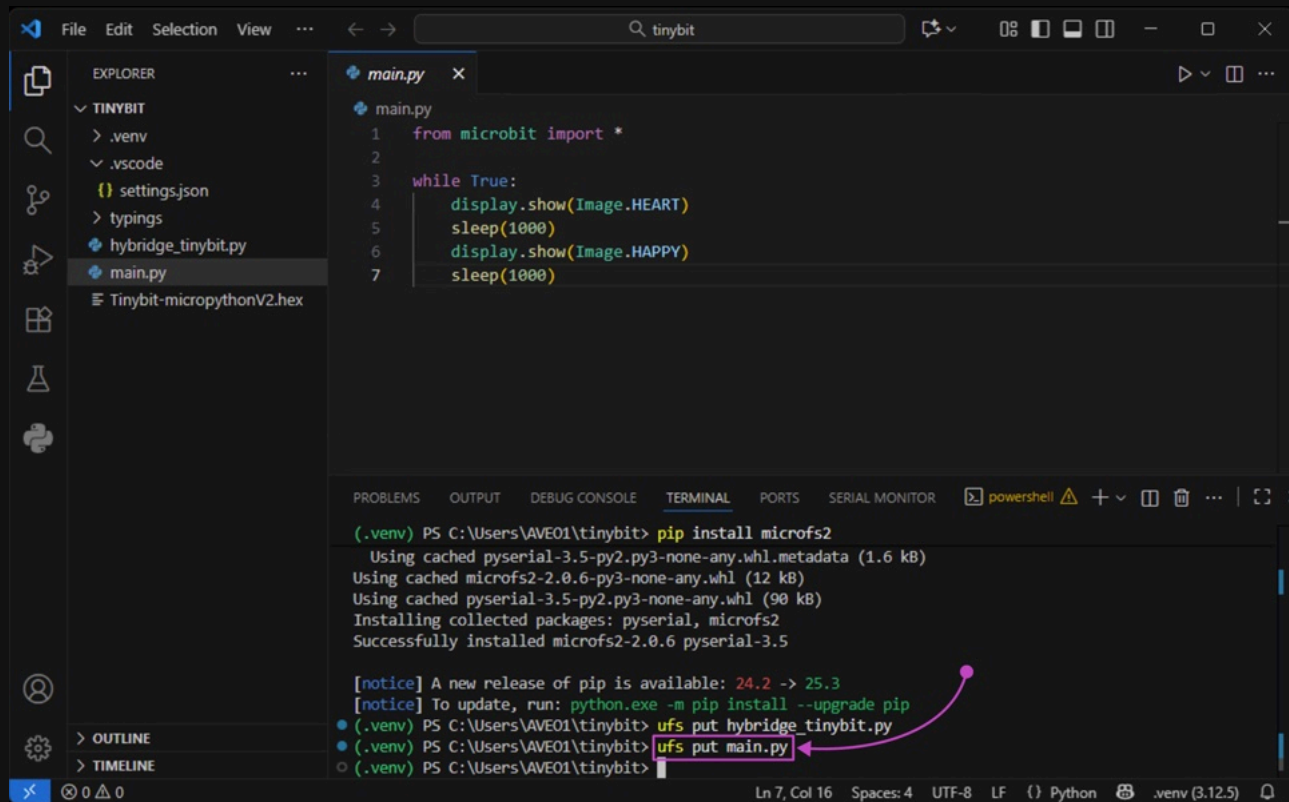
The TERMINAL panel at the bottom shows the following output:

```
(.venv) PS C:\Users\VAE01\tinybit> pip install microfs2
Collecting pyserial<4.0,>=3.0.1 (from microfs2)
  Using cached pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
Using cached microfs2-2.0.6-py3-none-any.whl (12 kB)
Using cached pyserial-3.5-py2.py3-none-any.whl (90 kB)
Installing collected packages: pyserial, microfs2
Successfully installed microfs2-2.0.6 pyserial-3.5

[notice] A new release of pip is available: 24.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\VAE01\tinybit> ufs put hybridge_tinybit.py
(.venv) PS C:\Users\VAE01\tinybit>
```

The status bar at the bottom indicates 'Ln 1, Col 1', 'Spaces: 4', 'UTF-8', 'LF', 'Python', and '.venv (3.12.5)'.

🐍 Put main.py on Micro:Bit



The screenshot shows the Visual Studio Code interface with a project named 'TINYBIT'. The Explorer sidebar on the left shows the file structure, including a 'main.py' file. The main editor window displays the code for 'main.py', which is a simple loop that shows a heart image and then a happy face image, each for 1000 milliseconds. Below the editor, the TERMINAL panel is open, showing the output of a PowerShell command to install the 'microfs2' package. The command 'ufs put main.py' is highlighted with a red box and a red arrow pointing to it from the text 'Put main.py on Micro:Bit' in the title bar. The status bar at the bottom indicates the current file is 'main.py' and the Python interpreter is set to '.venv (3.12.5)'.

```
File Edit Selection View ... < -> Q tinybit 0% 100% 100% - □ X
```

EXPLORER ...

- TINYBIT
 - .venv
 - .vscode
 - settings.json
 - typings
 - hybridge_tinybit.py
 - main.py
- Tinybit-micropythonV2.hex

main.py X

```
main.py
1 from microbit import *
2
3 while True:
4     display.show(Image.HEART)
5     sleep(1000)
6     display.show(Image.HAPPY)
7     sleep(1000)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR powershell ⚠ + - □ X ... | [] >

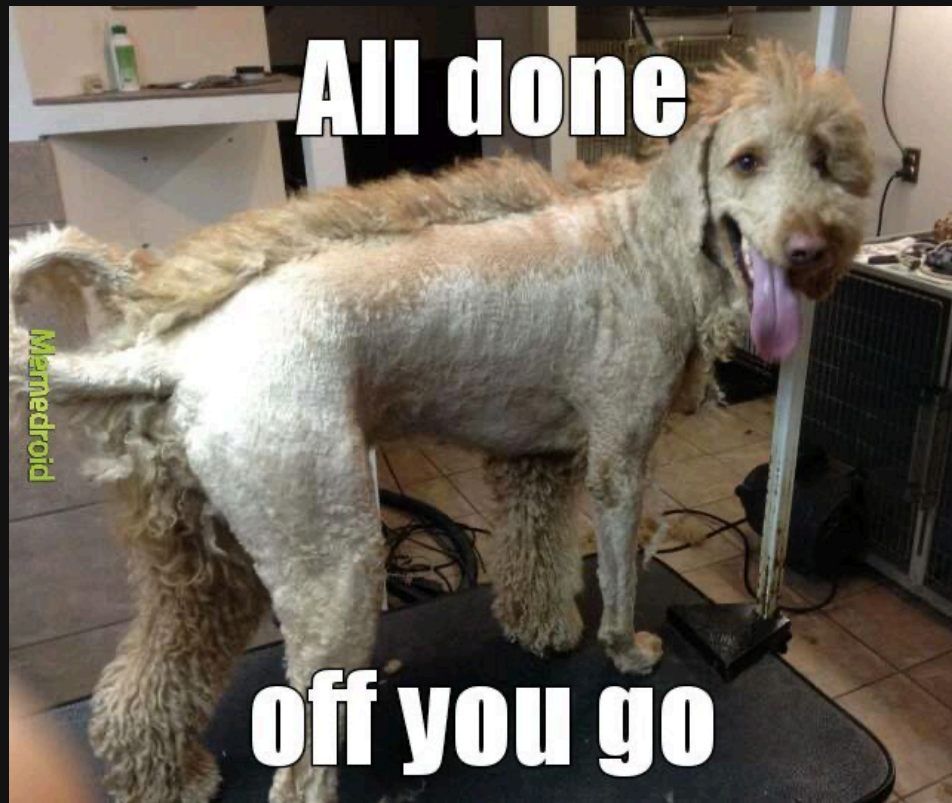
```
(.venv) PS C:\Users\AVE01\tinybit> pip install microfs2
Using cached pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
Using cached microfs2-2.0.6-py3-none-any.whl (12 kB)
Using cached pyserial-3.5-py2.py3-none-any.whl (90 kB)
Installing collected packages: pyserial, microfs2
Successfully installed microfs2-2.0.6 pyserial-3.5

[notice] A new release of pip is available: 24.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\AVE01\tinybit> ufs put hybridge_tinybit.py
(.venv) PS C:\Users\AVE01\tinybit> ufs put main.py
(.venv) PS C:\Users\AVE01\tinybit>
```

OUTLINE
TIMELINE

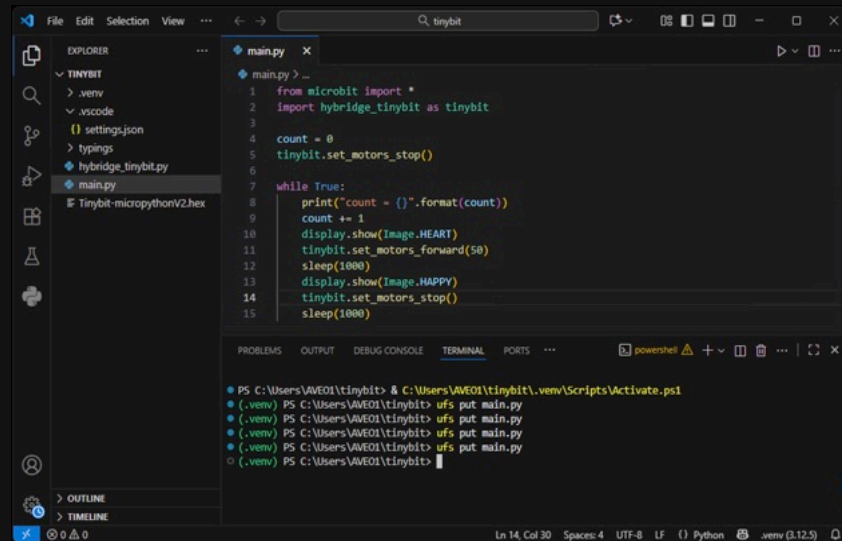
0 0 0 Ln 7, Col 16 Spaces: 4 UTF-8 LF {} Python .venv (3.12.5)

🐍 Yay!



🐍 Normal Development Flow

1. Modify `main.py`
2. Save `main.py`
3. If you need to plug in Micro:Bit USB programming port, wait 10 seconds afterwards
4. In your venv: `ufs put main.py`



The screenshot shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'TINYBIT' with subfolders '.venv', '.vscode', and files 'settings.json', 'typings', 'hybridge_tinybit.py', and 'main.py'. The 'main.py' file is open in the editor, showing the following code:

```
1 from microbit import *
2 import hybridge_tinybit as tinybit
3
4 count = 0
5 tinybit.set_motors_stop()
6
7 while True:
8     print("count = {}".format(count))
9     count += 1
10    display.show(Image.HEART)
11    tinybit.set_motors_forward(50)
12    sleep(1000)
13    display.show(Image.HAPPY)
14    tinybit.set_motors_stop()
15    sleep(1000)
```

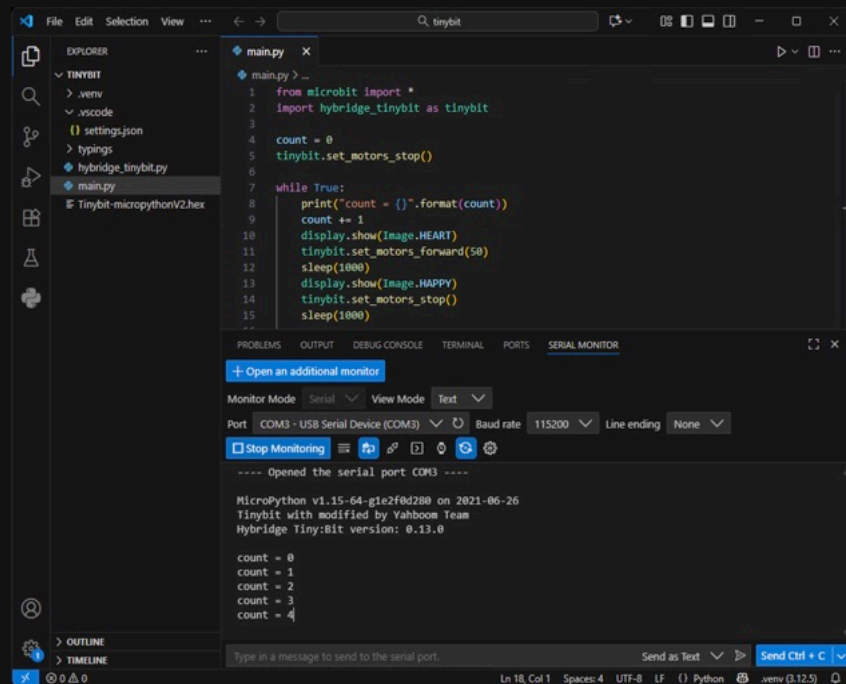
The terminal window at the bottom shows the following commands and output:

```
PS C:\Users\VAE01\tinybit> & C:\Users\VAE01\tinybit\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\VAE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAE01\tinybit>
```

The status bar at the bottom indicates the current file is 'main.py' at line 14, column 30, with 4 spaces, UTF-8 encoding, LF line endings, and Python syntax highlighting.

🐍 Debugging

- Option 1:
 - Micro:Bit display
 - Tiny:Bit LEDs
- Option 2:
 - Use `print`
 - View with the Serial Monitor extension
 - Have to keep USB cable attached



The screenshot shows the Visual Studio Code interface with a Python file named `main.py` open. The Explorer sidebar on the left shows the file structure for a project named 'TINYBIT', including files like `.venv`, `.vscode`, `settings.json`, `typings`, `hybridge_tinybit.py`, and `main.py`. The `main.py` file contains the following code:

```
1 from microbit import *
2 import hybridge_tinybit as tinybit
3
4 count = 0
5 tinybit.set_motors_stop()
6
7 while True:
8     print("count = {}".format(count))
9     count += 1
10    display.show(Image.HEART)
11    tinybit.set_motors_forward(50)
12    sleep(1000)
13    display.show(Image.HAPPY)
14    tinybit.set_motors_stop()
15    sleep(1000)
16 ..
```

Below the code editor, the SERIAL MONITOR extension is active, showing the output of the program. The output includes the MicroPython version, the Tinybit version, and the sequence of counts and motor actions:

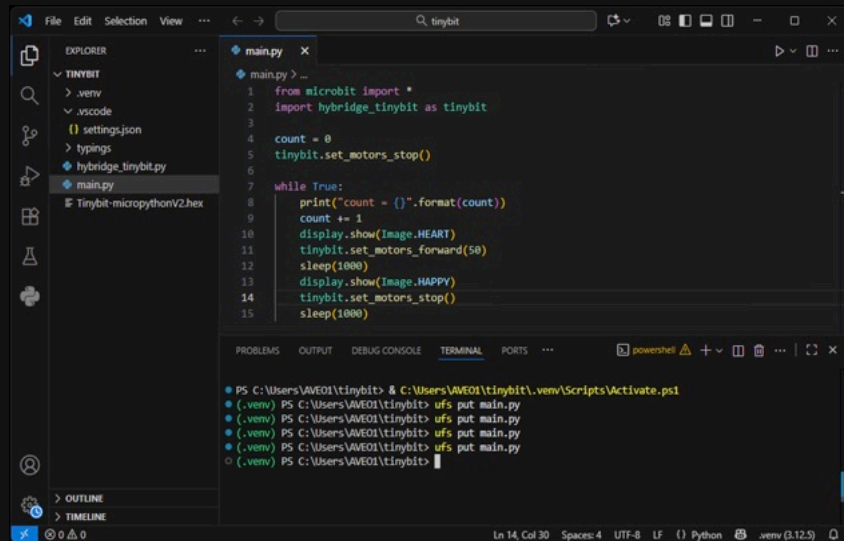
```
----- Opened the serial port COM3 -----
MicroPython v1.15-64-g1e2f0d280 on 2021-06-26
Tinybit with modified by Yahboom Team
Hybridge Tiny:bit version: 0.13.0

count = 0
count = 1
count = 2
count = 3
count = 4
```

The status bar at the bottom indicates the current position is Line 18, Column 1, with 4 spaces, UTF-8 encoding, LF line endings, Python language, and the .venv environment.

🐍 Common Problems

- Did you forget to save your `main.py` ?
- Is your serial monitor connected?
 - Can't `ufs` put `main.py` while connected
- Are you in your virtual environment?
 - You should see `(.venv)` left of your prompt
 - If not, `.venv\Scripts\Activate.ps1`
- Is the Tiny:Bit power switch on?
- Is your battery charged?
- Did you brick your Micro:Bit?



The screenshot shows the Visual Studio Code interface. The Explorer sidebar on the left shows a file tree for a project named 'TINYBIT' with files like `.venv`, `.vscode`, `settings.json`, `typings`, `hybridge_tinybit.py`, `main.py`, and `Tinybit-micropythonV2.hex`. The main editor window displays the `main.py` file with the following Python code:

```
1 from microbit import *
2 import hybridge_tinybit as tinybit
3
4 count = 0
5 tinybit.set_motors_stop()
6
7 while True:
8     print("count = {}".format(count))
9     count += 1
10    display.show(Image.HEART)
11    tinybit.set_motors_forward(50)
12    sleep(1000)
13    display.show(Image.HAPPY)
14    tinybit.set_motors_stop()
15    sleep(1000)
```

At the bottom, the TERMINAL panel shows a PowerShell session with the following commands and output:

```
PS C:\Users\VAWE01\tinybit> & C:\Users\VAWE01\tinybit\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\VAWE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAWE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAWE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAWE01\tinybit> ufs put main.py
(.venv) PS C:\Users\VAWE01\tinybit>
```

The status bar at the bottom indicates the current file is `main.py` at line 14, column 30, with 4 spaces, UTF-8 encoding, LF line endings, and the Python interpreter is set to `.venv (3.12.5)`.



If You Brick Your Micro:Bit

1. Turn off Tiny:Bit power switch
2. Unplug Micro:Bit and wait 10 seconds
3. Plug USB into Micro:Bit programming port and wait 10 seconds
4. Copy `Tinybit-micropythonV2.hex` to Micro:Bit USB Drive
5. In your venv: `ufs put hybridge_tinybit.py`
6. In your venv: `ufs put main.py`
7. It should work now



🐍 Tiny:Bit Lights

```
import hybridge_tinybit as tinybit

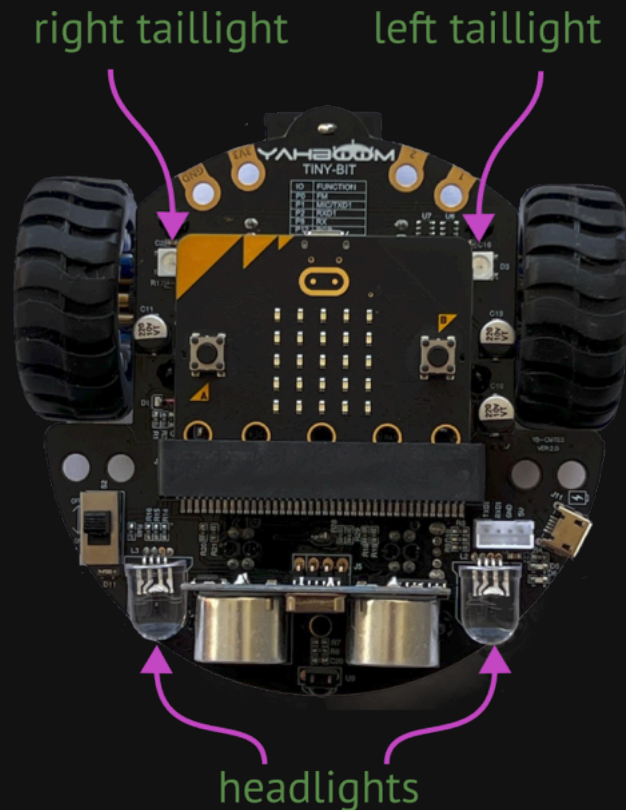
color_white_full = (255, 255, 255)
color_white_half = (128, 128, 128)
color_white_quarter = (64, 64, 64)
red = (64, 0, 0)
green = (0, 64, 0)
blue = (0, 0, 64)

tinybit.set_headlights(color_white_quarter)

tinybit.set_taillight_left(blue)
tinybit.set_taillight_right(green)

tinybit.set_taillights(blue, green)
```

- Colors set with RGB values (0-255) as tuple or list
- Brighter drains battery faster
- I like 64



Tiny:Bit IR Remote

```
import hybridge_tinybit as tinybit

ir_code = tinybit.get_ir_code()

if ir_code == tinybit.IR_LEFT:
    tinybit.set_motors_spin_left(50)
elif ir_code == tinybit.IR_LIGHT:
    tinybit.set_headlights((64, 64, 64))
elif ir_code != tinybit.IR_NO_SIGNAL:
    print("unhandled code {}".format(ir_code))
```

- Best reception from the front
- But kinda works from the back too
- The Micro:Bit's music interferes with the IR signal
 - Short sound effects are fine
 - Long songs cause problems
 - Codes get missed
 - Incorrect codes reported



infrared sensor



IR_POWER	IR_UP	IR_LIGHT
IR_LEFT	IR_CENTER	IR_RIGHT
IR_BACK	IR_DOWN	IR_FORWARD
IR_PLUS	IR_0	IR_MINUS
IR_1	IR_2	IR_3
IR_4	IR_5	IR_6
IR_7	IR_8	IR_9

🐍 Tiny:Bit Motors

```
import hybridge_tinybit as tinybit
```

```
# go forward with speed 50 (0-255)  
tinybit.set_motors_forward(50)
```

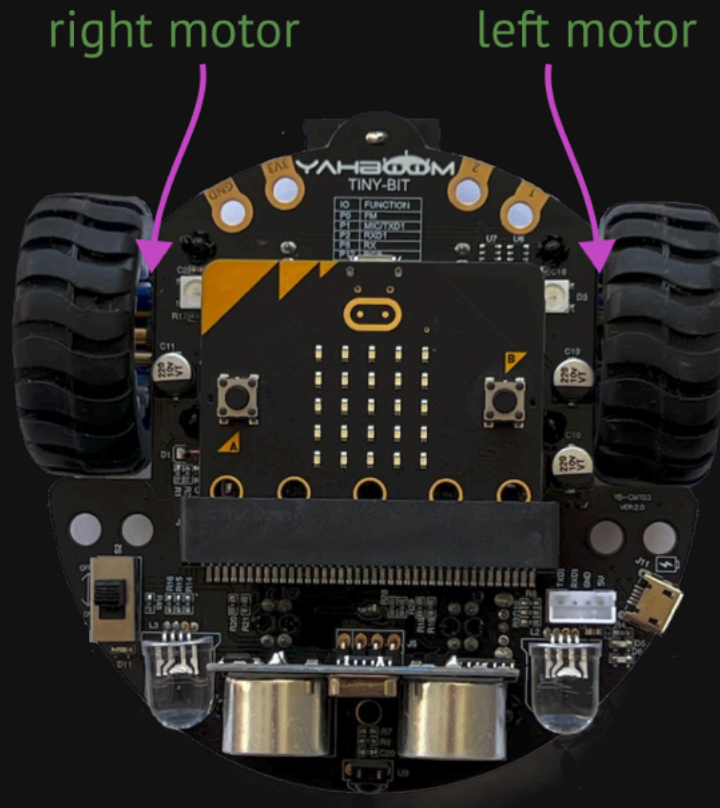
```
# go backward with speed 50 (0-255)  
tinybit.set_motors_backward(50)
```

```
# spin left with speed 50 (0-255)  
tinybit.set_motors_spin_left(50)
```

```
# spin right with speed 50 (0-255)  
tinybit.set_motors_spin_right(50)
```

```
# stop motors  
tinybit.set_motors_stop()
```

- 50 is a good speed for hard floors
- Will go slower as battery runs down
- Best battery life on hard floors
- Battery will wear down much faster on carpet

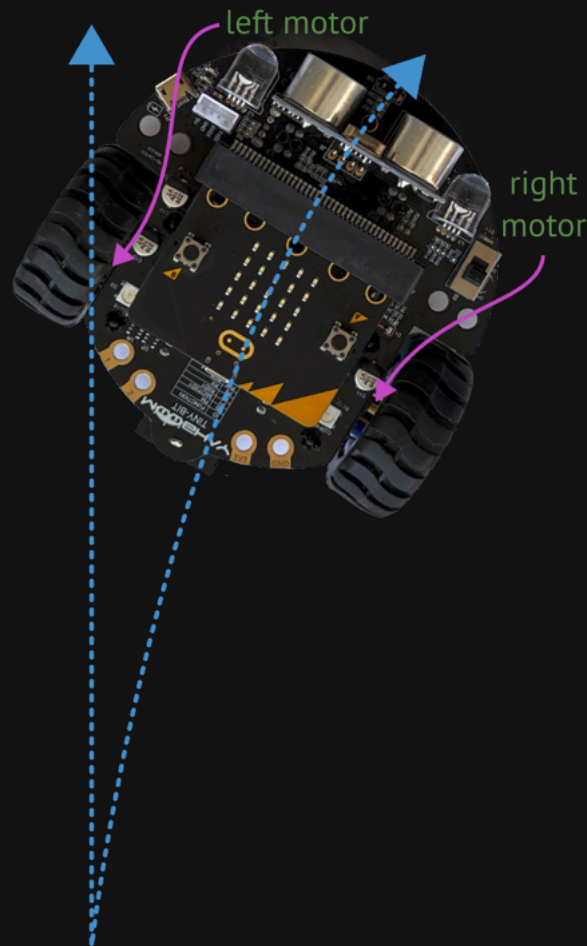


🐍 Tiny:Bit Motor Trim (Optional)

```
import hybridge_tinybit as tinybit

tinybit.set_motors_trim(3)
tinybit.set_motors_forward(50)
tinybit.set_motors_spin_left(50)
```

- Tiny:Bit can pull to one side
- Due to electrical and mechanical differences
- Can compensate with trim
- Positive: increase speed of left motor
- Negative: increase speed of right motor
- Set it once at top of `main.py`
- Gets applied to all other motor functions
- Drift will change as battery wears down
- Drift will change as motors wear in



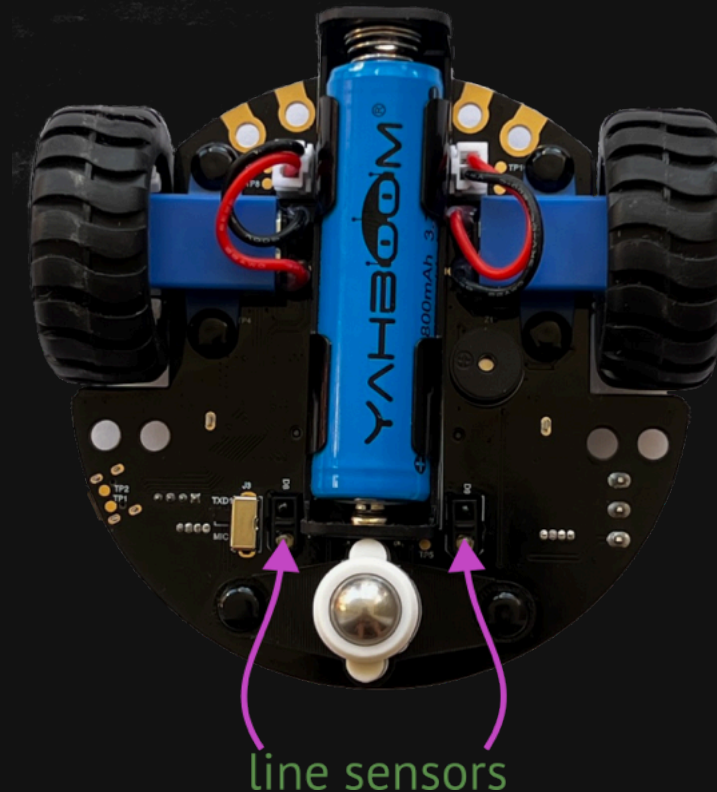
🐍 Tiny:Bit Line Sensors

```
import hybridge_tinybit as tinybit

red = (64, 0, 0)

if tinybit.has_left_track():
    tinybit.set_taillight_left(red)
elif tinybit.has_right_track():
    tinybit.set_taillight_right(red)
```

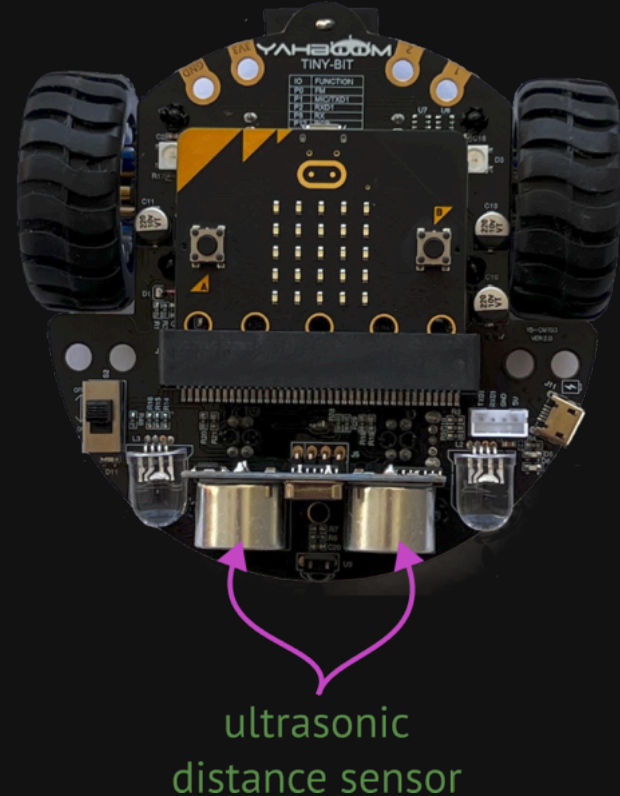
- IR emitter/detectors
- Detect black/white
 - `True` if black
 - `False` if white
- We will use for line following



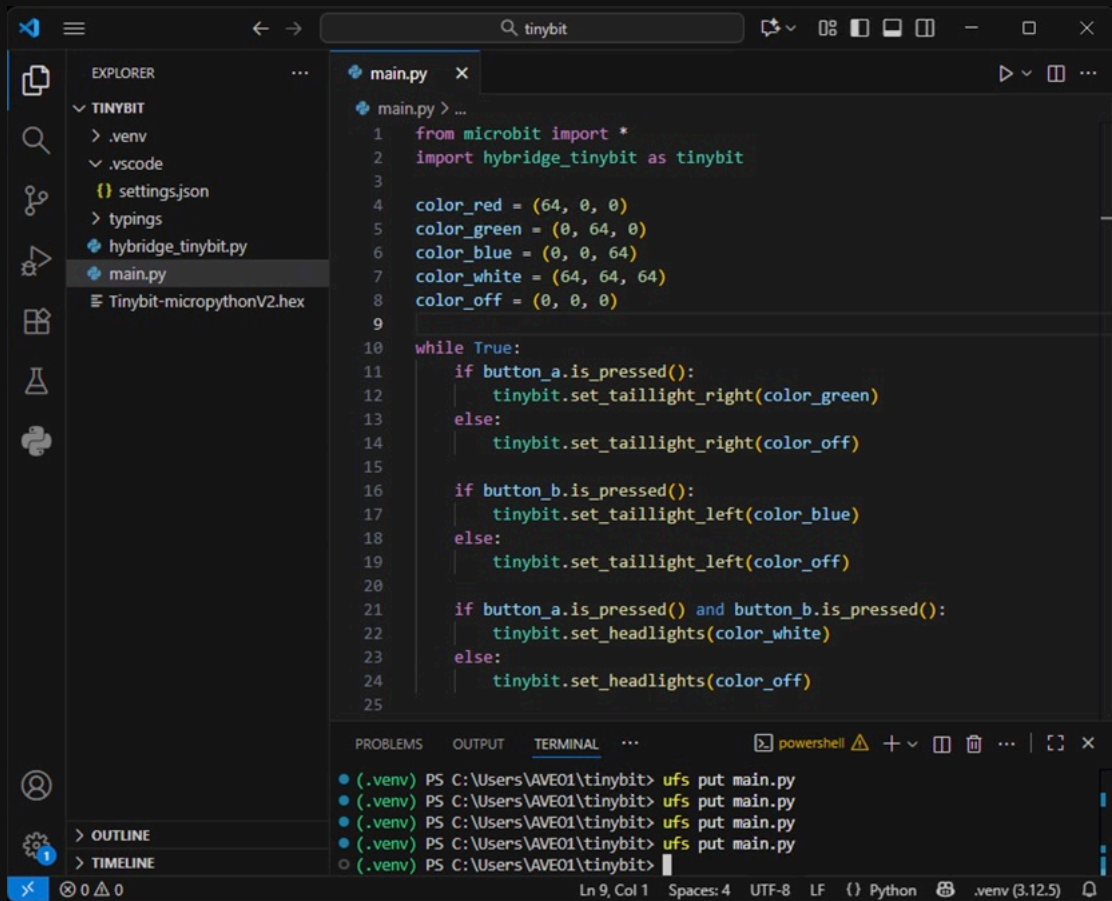
🐍 Tiny:Bit Ultrasonic Distance Sensor

```
import hybridge_tinybit as tinybit  
  
dist = tinybit.get_sonar_distance()
```

- Returns distance (in cm)
- Best results in 10 cm - 250 cm range
- Can give erroneous results outside that range
- Blind spot around 3 cm
- ~40° field of view



Sample Program



The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left displays the project structure for 'TINYBIT', including files like .venv, .vscode, settings.json, typings, hybridge_tinybit.py, main.py, and Tinybit-micropythonV2.hex. The main editor window is open to 'main.py', which contains the following Python code:

```
1 from microbit import *
2 import hybridge_tinybit as tinybit
3
4 color_red = (64, 0, 0)
5 color_green = (0, 64, 0)
6 color_blue = (0, 0, 64)
7 color_white = (64, 64, 64)
8 color_off = (0, 0, 0)
9
10 while True:
11     if button_a.is_pressed():
12         tinybit.set_taillight_right(color_green)
13     else:
14         tinybit.set_taillight_right(color_off)
15
16     if button_b.is_pressed():
17         tinybit.set_taillight_left(color_blue)
18     else:
19         tinybit.set_taillight_left(color_off)
20
21     if button_a.is_pressed() and button_b.is_pressed():
22         tinybit.set_headlights(color_white)
23     else:
24         tinybit.set_headlights(color_off)
25
```

The bottom of the interface shows the TERMINAL panel with a PowerShell session. It contains five identical lines of command and output:

```
(.venv) PS C:\Users\AVE01\tinybit> ufs put main.py
(.venv) PS C:\Users\AVE01\tinybit> ufs put main.py
(.venv) PS C:\Users\AVE01\tinybit> ufs put main.py
(.venv) PS C:\Users\AVE01\tinybit> ufs put main.py
(.venv) PS C:\Users\AVE01\tinybit>
```

The status bar at the very bottom indicates the current cursor position (Ln 9, Col 1), indentation (Spaces: 4), encoding (UTF-8), line endings (LF), language (Python), and the active environment (.venv [3.12.5]).



Homework

- Control the Tiny:Bit with the IR remote
 - Must at least:
 - Do things when IR remote buttons are pressed
 - Do something with the headlights
 - Do something with the taillights
 - Do something with the motors
 - Do something with the Micro:Bit display
 - Have fun!
 - Bring it next time to demo
- Optional ideas:
 - Play sounds with the Micro:Bit
 - Use Micro:Bit buttons
 - Detect tilt with Micro:Bit
 - Detect gestures with Micro:Bit
 - Detect light levels with Micro:Bit
 - Detect distance with Tiny:Bit ultrasonic sensor